

How DI Is Performed

3 WAYS TO INJECT A DEPENDENCY

1

Constructor Injection

DEFAULT CHOICE

```
class OrderService {
    private final PaymentGateway payment;

    OrderService(PaymentGateway payment) {
        this.payment = payment;
    }
}
```

Dependencies arrive as constructor parameters. The object can never exist in a half-configured state — if it exists, its dependencies are set.

2

Setter Injection

FOR OPTIONAL DEPS

```
class OrderService {
    private PaymentGateway payment;

    void setPaymentGateway(PaymentGateway p) {
        this.payment = p;
    }
}
```

Dependencies are set via a public setter after construction. Useful for optional dependencies, but the object can briefly exist unconfigured.

3

Interface Injection

RARELY USED TODAY

```
interface PaymentAware {
    void injectPayment(
        PaymentGateway p);
}

class OrderService
    implements PaymentAware {
    public void injectPayment(
        PaymentGateway p) {
        this.payment = p;
    }
}
```

The class implements an interface exposing an inject method, which a framework calls. More ceremony, so it has mostly fallen out of favor.

Which one should you use? Constructor injection is the default in modern frameworks — it makes required dependencies explicit and impossible to forget. Reach for setter injection only when a dependency is genuinely optional.